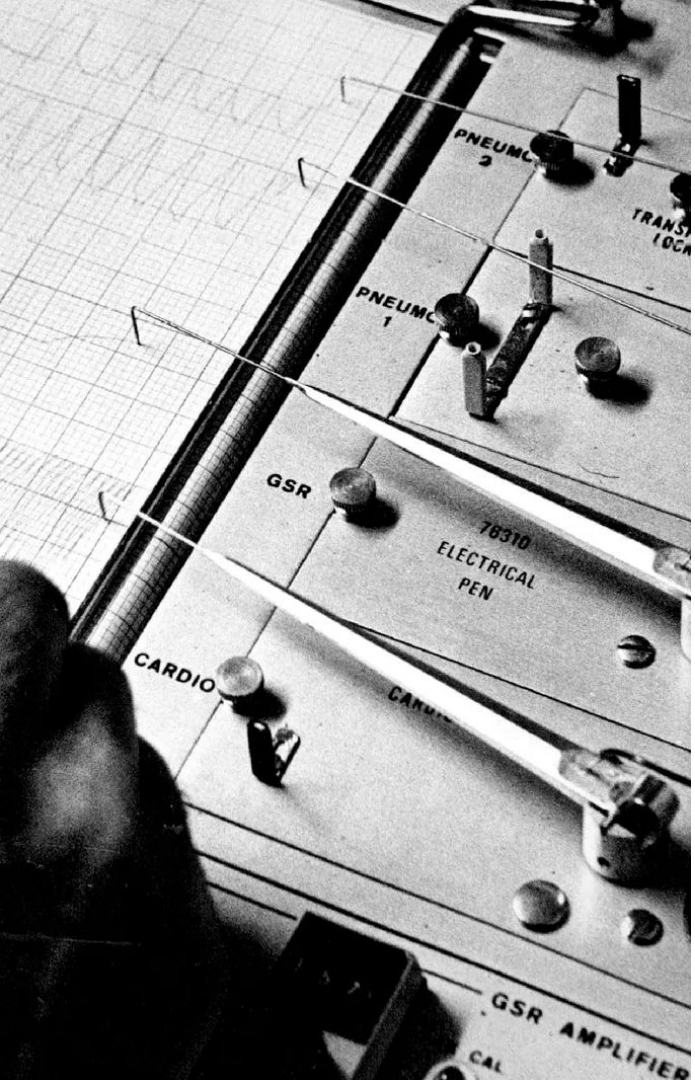


CheckIt.AI

Extrayez des données multimodales de sites web

Concevez une stratégie d’extraction de données multimodales pour entraîner un détecteur de fake news

Rafael CEREZO MARTIN

Juin 2026

Présentation

Détection de Fake News · Architecture Hexagonale · Apache Airflow

CheckIt.AI

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️TransformService
- ⚙️Pipeline Service

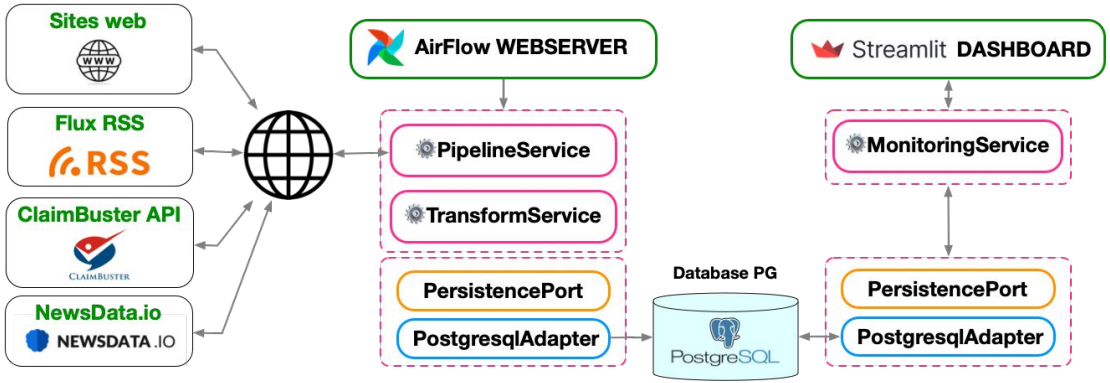
4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️Monitoring Service
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Un pipeline ETL multimodal pour détecter la désinformation

Pipeline d'extraction de données multimodales (texte + image) , transformation et orchestration de publications, pour **l'entraînement d'un détecteur de fake news**, orchestré via Apache Airflow selon une architecture hexagonale (Ports & Adapters) qui isole le domaine métier de la technique.



STACK TECHNIQUE

Python 3.12 · Airflow 2.10.5 · Scrapy latest · Selenium 4 · PostgreSQL latest · Streamlit latest

Architecture hexagonale (Ports & Adapters)

CheckIt.AI

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

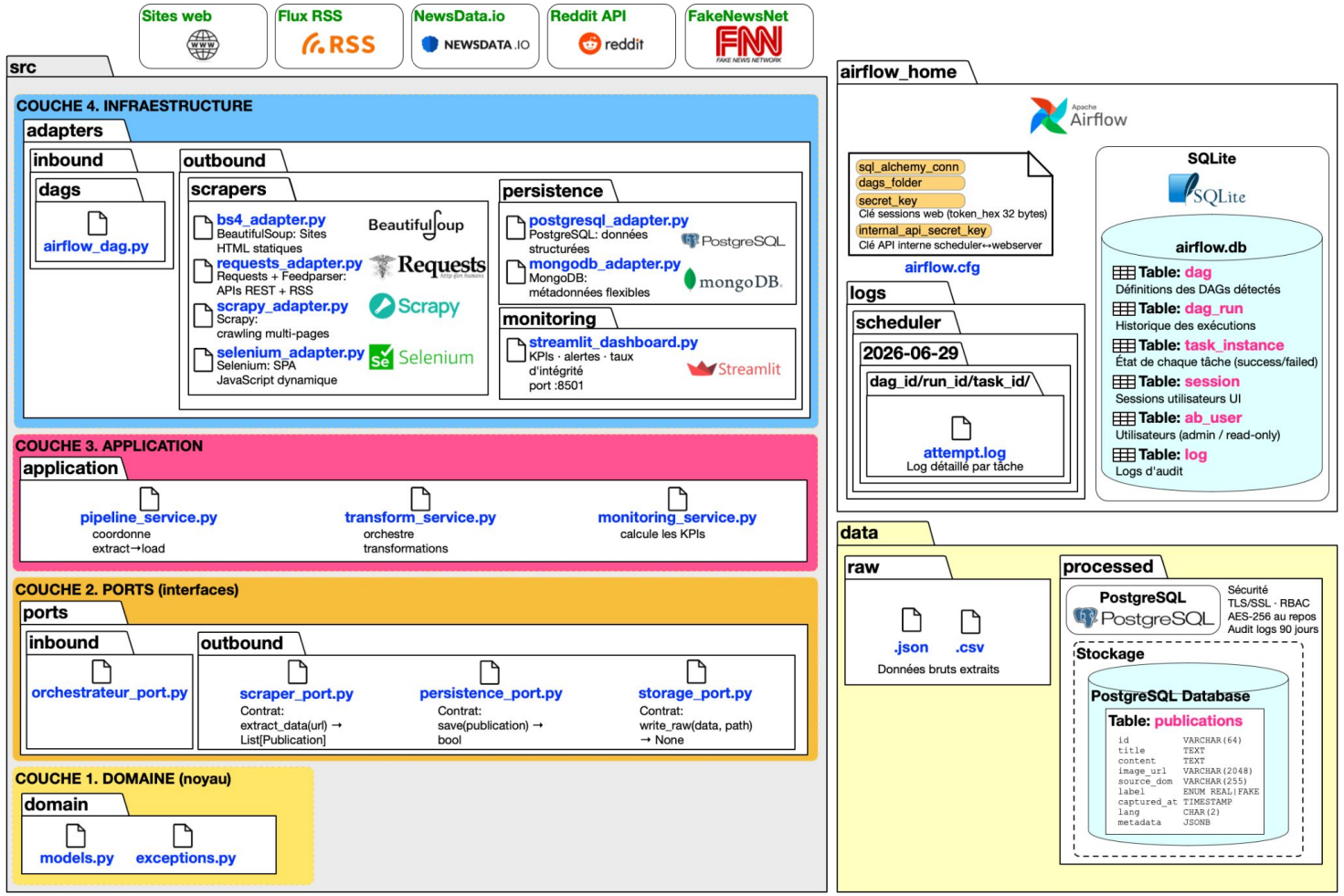
2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion



Architecture hexagonale (Domain Layer & Ports)

CheckIt.AI

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

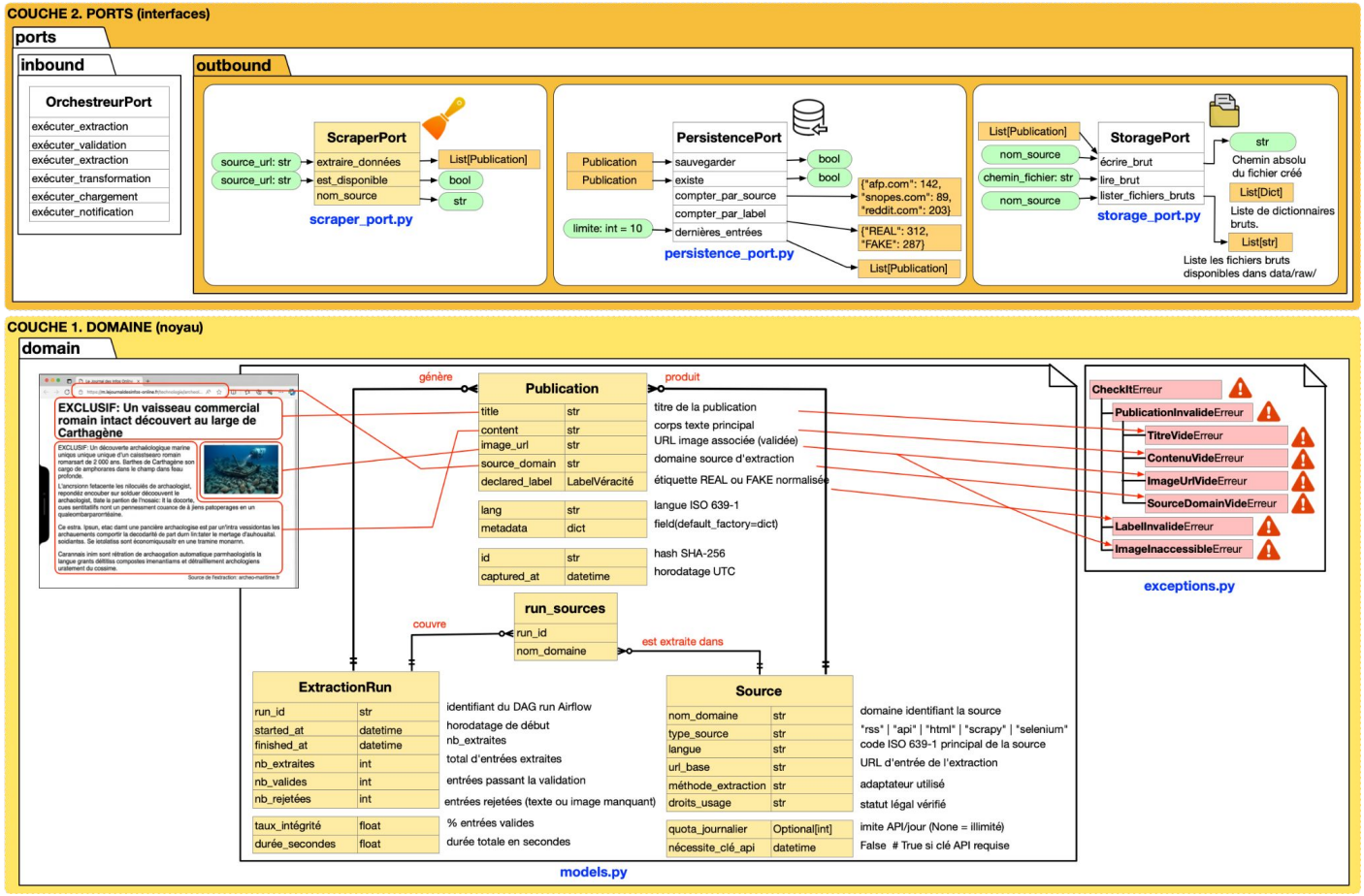
2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion



CheckIt.AI

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

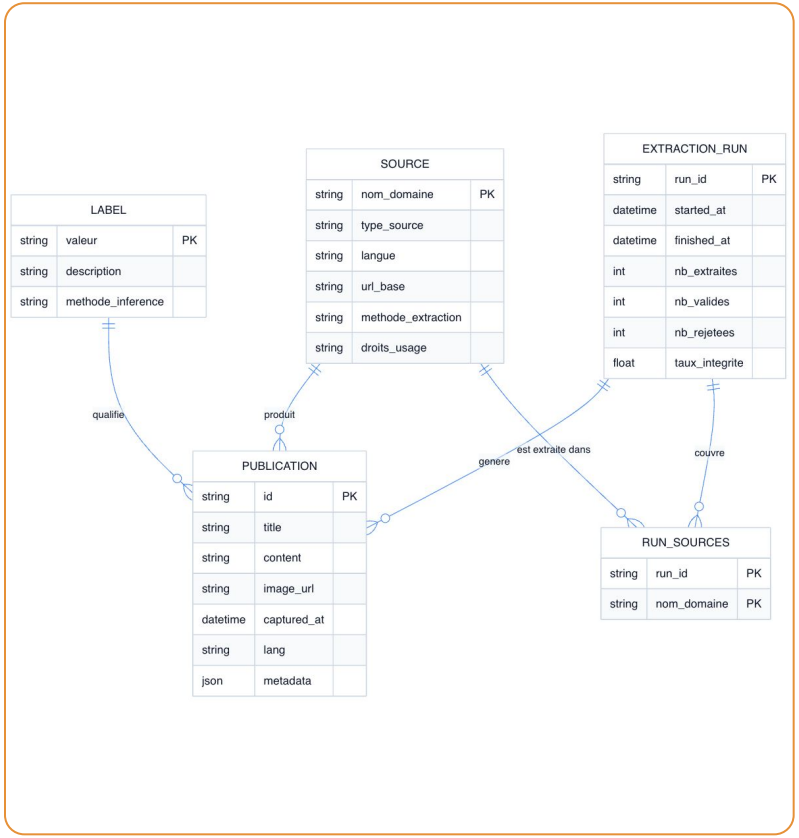
3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

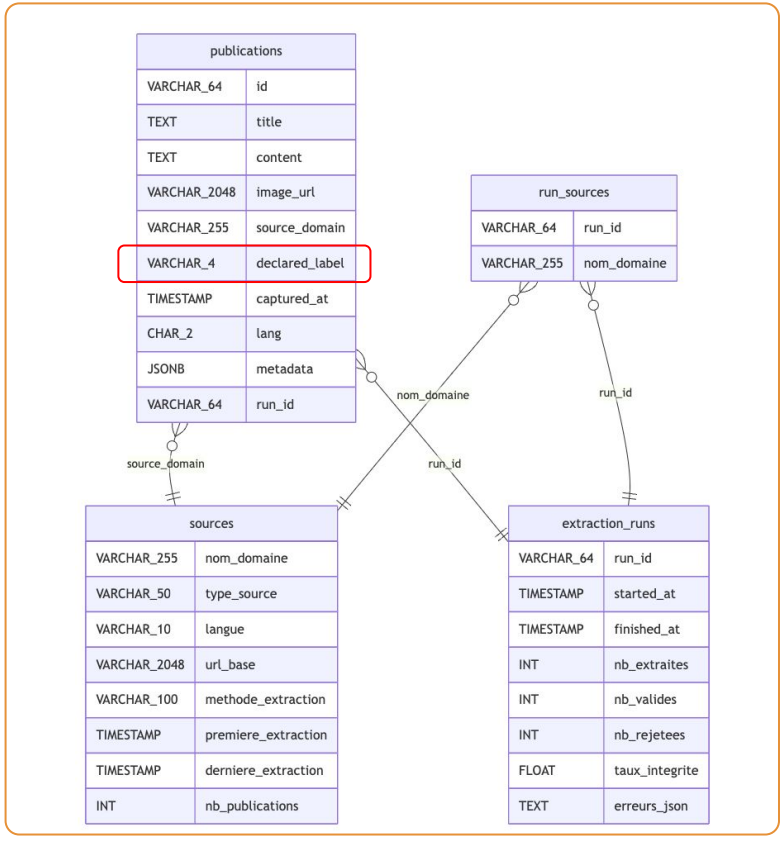
6. Conclusion

Modèle conceptuel



Représentation métier indépendante de toute technologie.
Langage compréhensible par les parties prenantes non techniques.

Modèle physique



Structure SQL concrète avec types, index et contraintes d'intégrité.

CheckIt.AI

1

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Sources utilises pour chaque adaptateur

Toutes les sources de labellisation retenues (AFP Factuel, EUvsDisinfo, PolitiFact, Correctiv, Maldita, FullFact, MediaBiasFactCheck, ClaimBuster) sont des organismes de fact-checking reconnus, avec une méthodologie de vérification publique et documentée.

Adaptateur		Source	Langue	Labels	
feedparser Feedparser	AFP Factuel	AFP Factuel	FR	Explicites titre	RSS
feedparser Feedparser	EUvsDisinfo	EUvsDisinfo	EN	Tous FAKE	
feedparser Feedparser	Les Observateurs	Les Observateurs	FR	3 classes	
feedparser Feedparser	Hoaxbuster	Hoaxbuster	FR	Explicites	
Requests Requests	NEWSDATA.IO	NewsData.io	Multi	Via MBFC	API
Requests Requests	ClaimBuster	ClaimBuster	Multi	Enrichissement score	
Requests Requests	Media Bias / Fact Check	MBFC	Multi	Inférence label	
BeautifulSoup BS4	FULL FACT	FullFact UK	EN	4 classes	HTML Statique
BeautifulSoup BS4	CORRECTIV	Correctiv DE	DE	3 classes	
BeautifulSoup BS4	MALDITA.ES	Maldita ES	ES	4 classes	
Scrapy Scrapy	POLITIFACT	PolitiFact	EN	6 classes	Pagination
Scrapy Scrapy	LES SURLIGNEURS	Les Surligneurs	FR	5 classes	
Selenium Selenium	Logically	Logically	EN	3 classes	JS dynamique
Selenium Selenium	Le Monde.fr	Decodex	FR	3 classes	

Étape 1. Explorez et qualifiez les sources de données

Modes de labellisation selon la source.

*Deux **mécanismes de labellisation** : mots-clés explicites (majorité) vs. inférence par réputation de domaine (NewsData.io, MBFC*).*

Variable: declared_label

Source	REAL ✅	FAKE ❌
AFP Factuel	`vrai` `vérifié` `confirmé`	`faux` `fake` `trompeur` `inexact` `manipulé` `hors contexte`
EUvsDisinfo	—	`disinformation` `fake` `false` `misleading` *(tous FAKE)*
Les Observateurs	`vrai` `vérifié`	`faux` `trompeur`
Hoaxbuster	`vrai` `vérifié`	`faux` `hoax` `trompeur`
NewsData.io	*(source fiable MBFC)* `bbc` `reuters` `afp` `lemonde`	*(source non fiable MBFC)* `rt` `sputnik` `breitbart` `infowars`
ClaimBuster	—	—
MBFC	`very high` `high`	`mixed` `low` `very low` `questionable` `conspiracy` `pseudoscience` `satire`
FullFact UK	`true` `correct` `mostly true`	`false` `misleading` `incorrect` `unverified` `missing context`
Correctiv DE	`richtig` `wahr` `stimmt`	`falsch` `irreführend` `fake`
Maldita ES	`verdadero` `verdad`	`falso` `engañoso` `bulo` `satira`
PolitiFact	`true` `mostly true`	`half-true` `mostly false` `false` `pants on fire`
Les Surligneurs	`exact` `vrai`	`inexact` `faux` `trompeur` `exagéré`
Logically	`true` `verified`	`false` `misleading` `unverified` `partly false`
Decodex	`vrai` `vérifié`	`faux` `trompeur` `inexact`

* **MBFC** = **Media Bias/Fact Check** . Organisme de fact-checking

CheckIt.AI

2

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

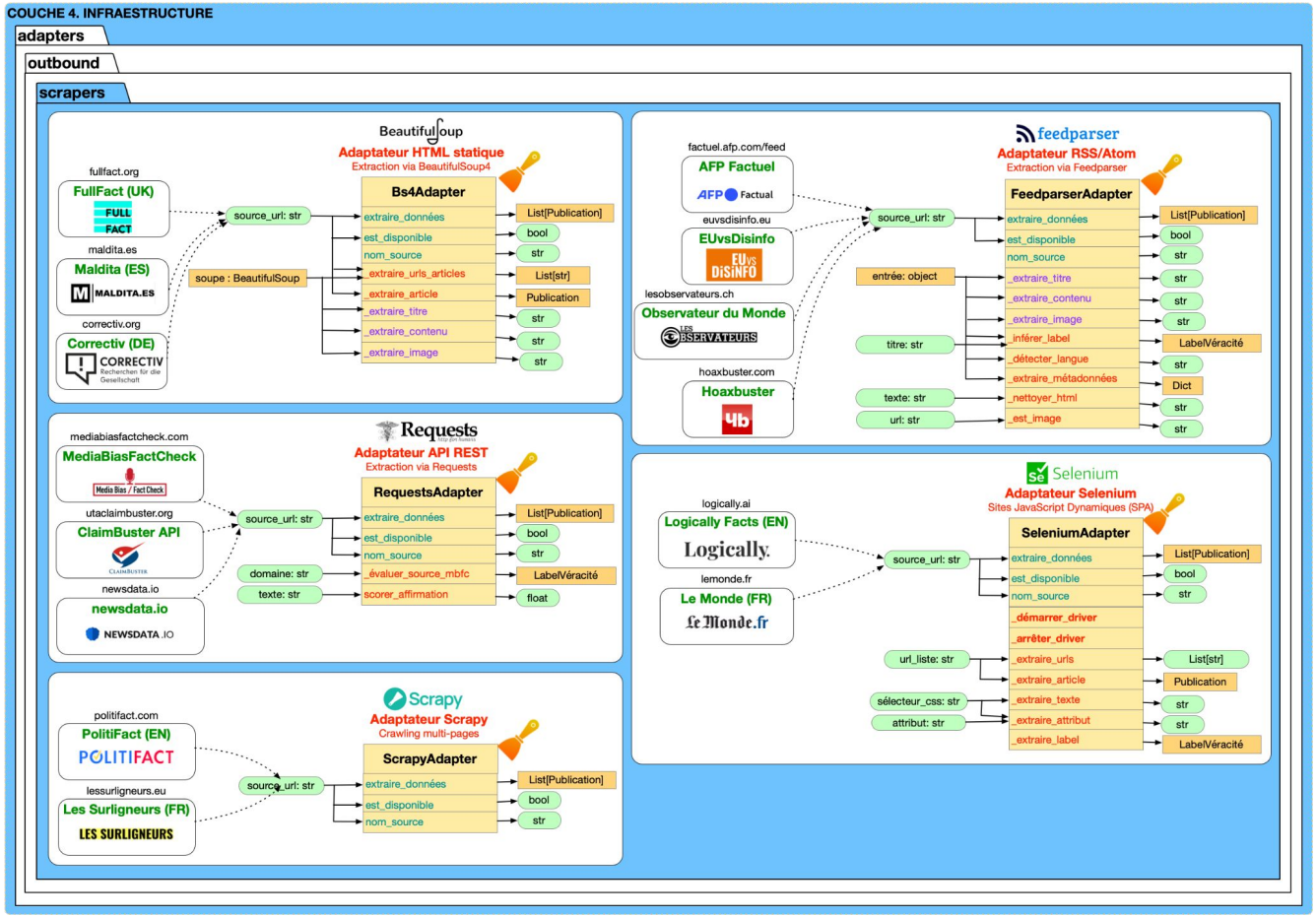
4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Adapters implémentés et méthodes specifiques

- *Cinq adaptateurs concrets, chacun implémentant le même contrat **ScraperPort** : **extraire_données, est_disponible, nom_source**.*
- *La signature est identique partout. C'est ce qui permet à **PipelineService** de les appeler sans savoir lequel tourne derrière.*



CheckIt.AI

2

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

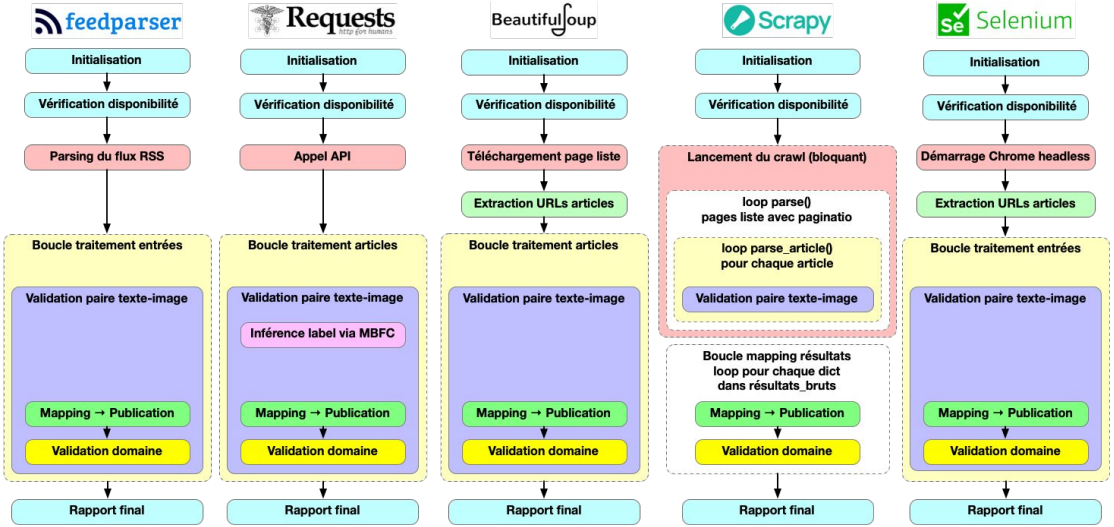
5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

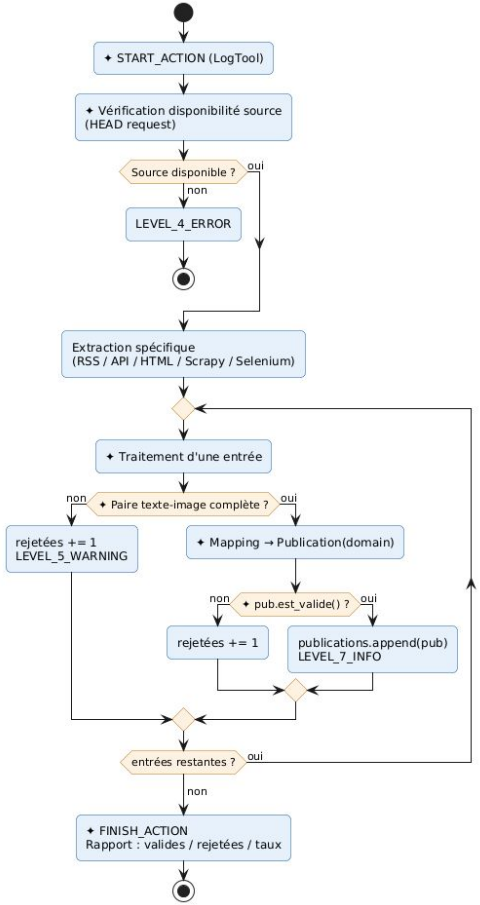
6. Conclusion

Étape 2. Développez des scripts d'extraction automatisée

Diagramme de séquence des Adaptateurs.
Etapes en commun et spécifiques.

- *Chaque adaptateur suit le même flux : vérification de disponibilité, extraction, validation de la paire texte-image, mapping vers Publication.*
- *Identification des risques réels*





⚠️Risques identifiés à surveiller

Spécifiques à Feedparser

- Flux RSS mal formé
- Flux RSS abandonné/déplacé sans redirection

Spécifiques aux APIs

- Clé API expirée ou révoquée
- Changement de schéma de réponse JSON
- API définitivement arrêtée
- Latence anormale de l'API

Communs à tous

- Changement de structure HTML
- Blocage par rate-limiting / robots.txt
- Changement de User-Agent requis
- Certificat SSL expiré ou invalide
- Redirection permanente (301) vers une nouvelle URL

Spécifiques à Selenium

- Timeout d'attente d'éléments
- Version incompatible de ChromeDriver
- Consommation mémoire excessive

CheckIt.AI

2

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Étape 2. Développez des scripts d'extraction automatisée

Résultats des tests: 5 adaptateurs

ADAPTATEUR	SOURCE TESTÉE	PUBLICATIONS	ÉTAT
Feedparser	AFP Factuel (Bluesky)	**5**	**16,7 % d'intégrité — images absentes**
Feedparser	EUvsDisinfo	**10**	**OK — 100 % d'intégrité**
Feedparser	Hoaxbuster	**9**	**OK — 100 % d'intégrité**
Feedparser	El País	**143**	**OK — 97,3 % d'intégrité**
Feedparser	PolitiFact (RSS) · Checked	**0**	**Labels non normalisables**
Feedparser	Les Décodeurs	**1**	**5,0 % d'intégrité — labels non reconnus**
Requests	NewsData.io	**10**	**OK**
Requests	Mediastack · ClaimBuster	**0**	**Quota dépassé (429) · indisponible (404)**
BeautifulSoup	FullFact UK · Correctiv DE · Maldita ES	**47**	**OK — toutes opérationnelles**
Scrapy	PolitiFact	**—**	**Log tronqué — à revérifier**
Selenium	AfricaCheck · Decodex (Le Monde)	**0**	**Sélecteurs CSS à ajuster**

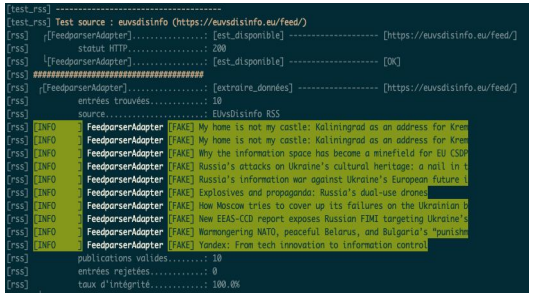
Exécution du 23/07/2026 · 10:17–10:22 UTC

225 publications extraites au total (hors Scrapy)

6 sources pleinement opérationnelles

5 sources à corriger: labels, quota ou sélecteurs

feedparser
Adaptateur RSS/Atom
Extraction via Feedparser



Requests
Adaptateur API REST
Extraction via Requests



BeautifulSoup
Adaptateur HTML statique
Extraction via BeautifulSoup4



CheckIt.AI

3

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

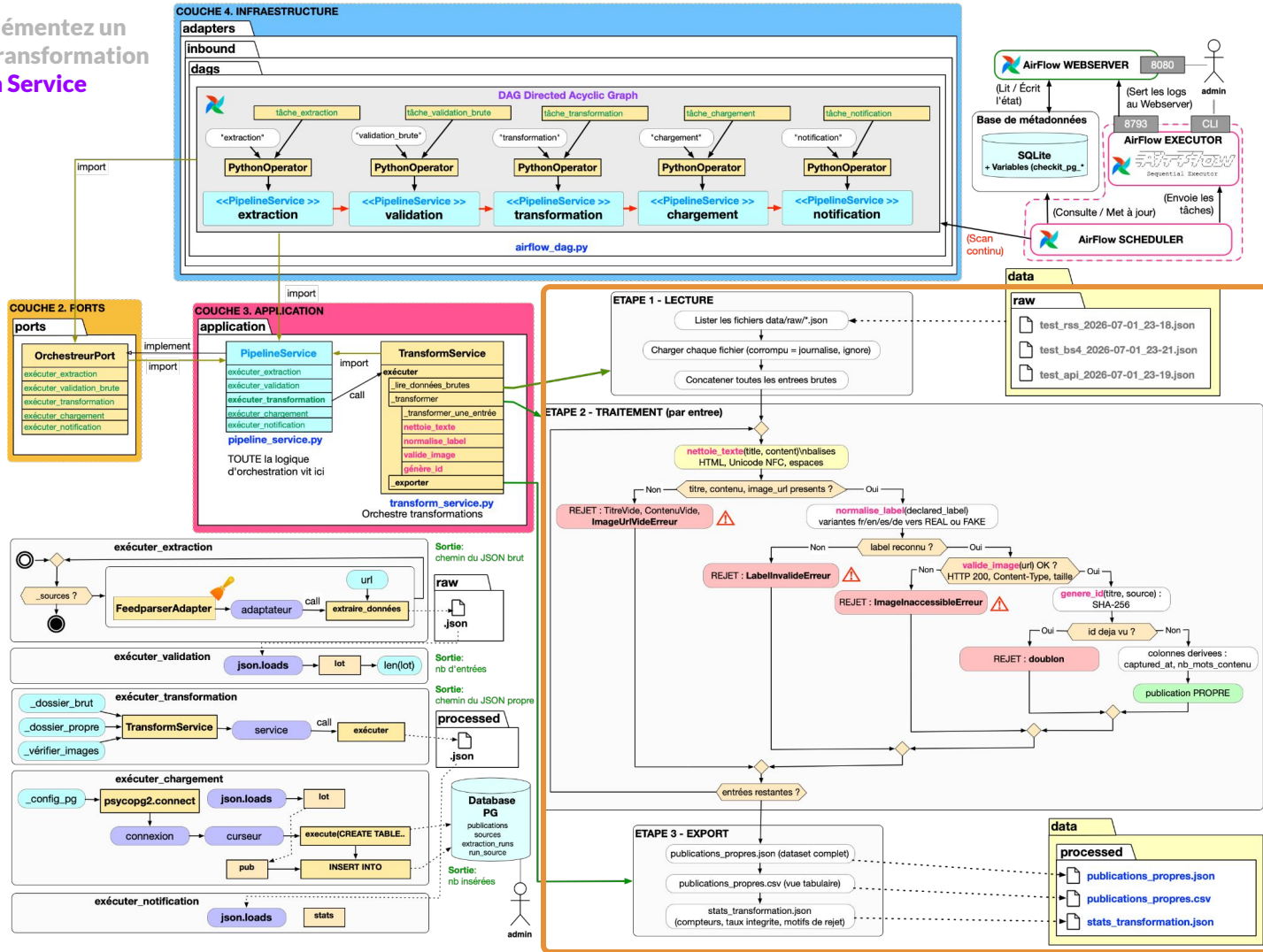
4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Étape 3. Implémentez un pipeline de transformation

⚙️Transform Service



CheckIt.AI

3

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

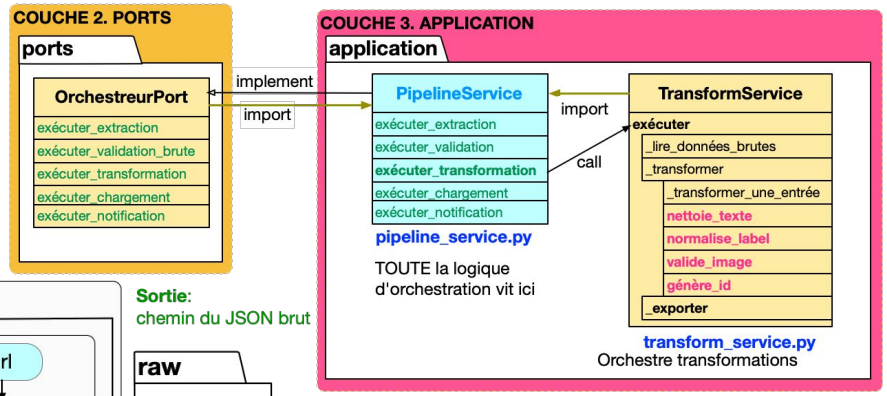
5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

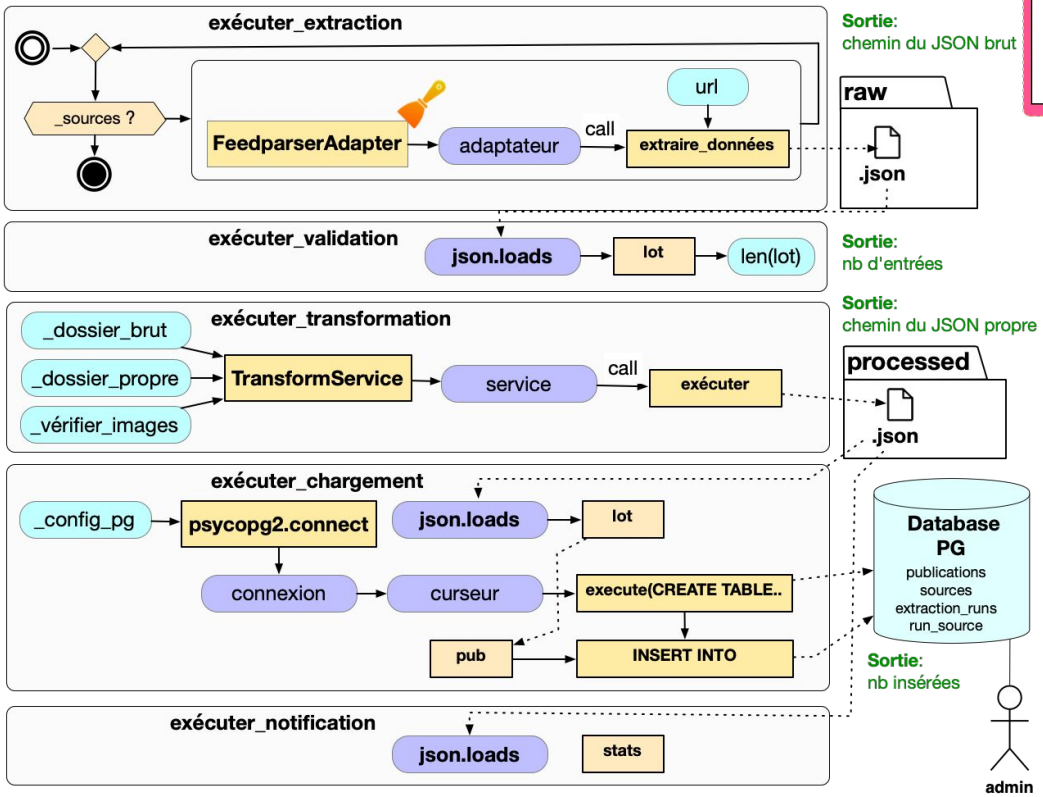
6. Conclusion

Étape 3. Implémentez un pipeline de transformation

Pipeline complet exécutable hors Airflow

⚙️**Pipeline Service**





Étape 3. Implémentez un pipeline de transformation

Pipeline complet exécutable hors Airflow

*Le pipeline complet est exécutable hors Airflow grâce au composition root CLI de **pipeline_service.py**, qui charge le .env via python-dotenv*

data/raw/*.json


```
[
  {
    "title": "Un porte-drapeau sud-africain tombe lors de la cérémonie d'ouverture du Mondial 2026 ? Très partagée, cette vidéo a très probablement été générée par IA pour
alimenter les tensions en Afrique du Sud, alors que des manifestants réclament depuis des mois le départ d'immigrés clandestins.",
    "content": "Un porte-drapeau sud-africain tombe lors de la cérémonie d'ouverture du Mondial 2026 ? Très partagée, cette vidéo a très probablement été générée par IA pour
alimenter les tensions en Afrique du Sud, alors que des manifestants réclament depuis des mois le départ d'immigrés clandestins.",
    "image_url": "https://video.bsky.app/watch/did%3Aplc%3A4ks5wkubjfcbgxvphqkd3wxm/bafkreigwnsf5iyrxc5e6pmacxhltp2bwcdbfbayia44tyq5uxsso5pe514/thumbnail.jpg",
    "source_domain": "afp_bluesky.com",
    "declared_label": "FAKE",
    "lang": "en",
    "metadata": {
      "url_source": "https://bsky.app/profile/factuel.afp.com/post/3mpgu475ivk2o",
      "auteur": "",
      "date_publi": "29 Jun 2026 15:57 +0000",
      "catégories": []
    },
    "id": "6ac90fbfa48d8b46c01be89a2c64b8ad596ce687a481ff2ab79a4222872e16f5",
    "captured_at": "2026-07-04 05:02:30.959076"
  },
```


uv run python3 src/application/pipeline_service.py →

```
RAPPORT DE TRANSFORMATION — ÉTAPE 3
  Entrées brutes......: 4607
  Publications valides: 1480
  Taux d'intégrité....: 32.1%
  REAL / FAKE.........: 1321 / 159
  Insérées en base....: 195
```

data/processed/publications_propres.json


```
{
    "id": "904d891d3a76111981b0ba4543a94d65e448a5621ea1d6b2bb9dc2a872ce2d26",
    "title": "Un porte-drapeau sud-africain tombe lors de la cérémonie d'ouverture du Mondial 2026 ? Très partagée, cette vidéo a très probablement été générée par IA pour alimenter les
tensions en Afrique du Sud, alors que des manifestants réclament depuis des mois le départ d'immigrés clandestins.",
    "content": "Un porte-drapeau sud-africain tombe lors de la cérémonie d'ouverture du Mondial 2026 ? Très partagée, cette vidéo a très probablement été générée par IA pour alimenter
les tensions en Afrique du Sud, alors que des manifestants réclament depuis des mois le départ d'immigrés clandestins.",
    "image_url": "https://video.bsky.app/watch/did%3Aplc%3A4ks5wkubjfcbgxvphqkd3wxm/bafkreigwnsf5iyrxc5e6pmacxhltp2bwcdbfbayia44tyq5uxsso5pe514/thumbnail.jpg",
    "source_domain": "afp_bluesky.com",
    "declared_label": "FAKE",
    "lang": "en",
    "captured_at": "2026-07-04 05:02:30.959076",
    "nb_mots_contenu": 44,
    "metadata": {
      "url_source": "https://bsky.app/profile/factuel.afp.com/post/3mpgu475ivk2o",
      "auteur": "",
      "date_publi": "29 Jun 2026 15:57 +0000",
      "catégories": []
    }
  },
```

CheckIt.AI

4

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

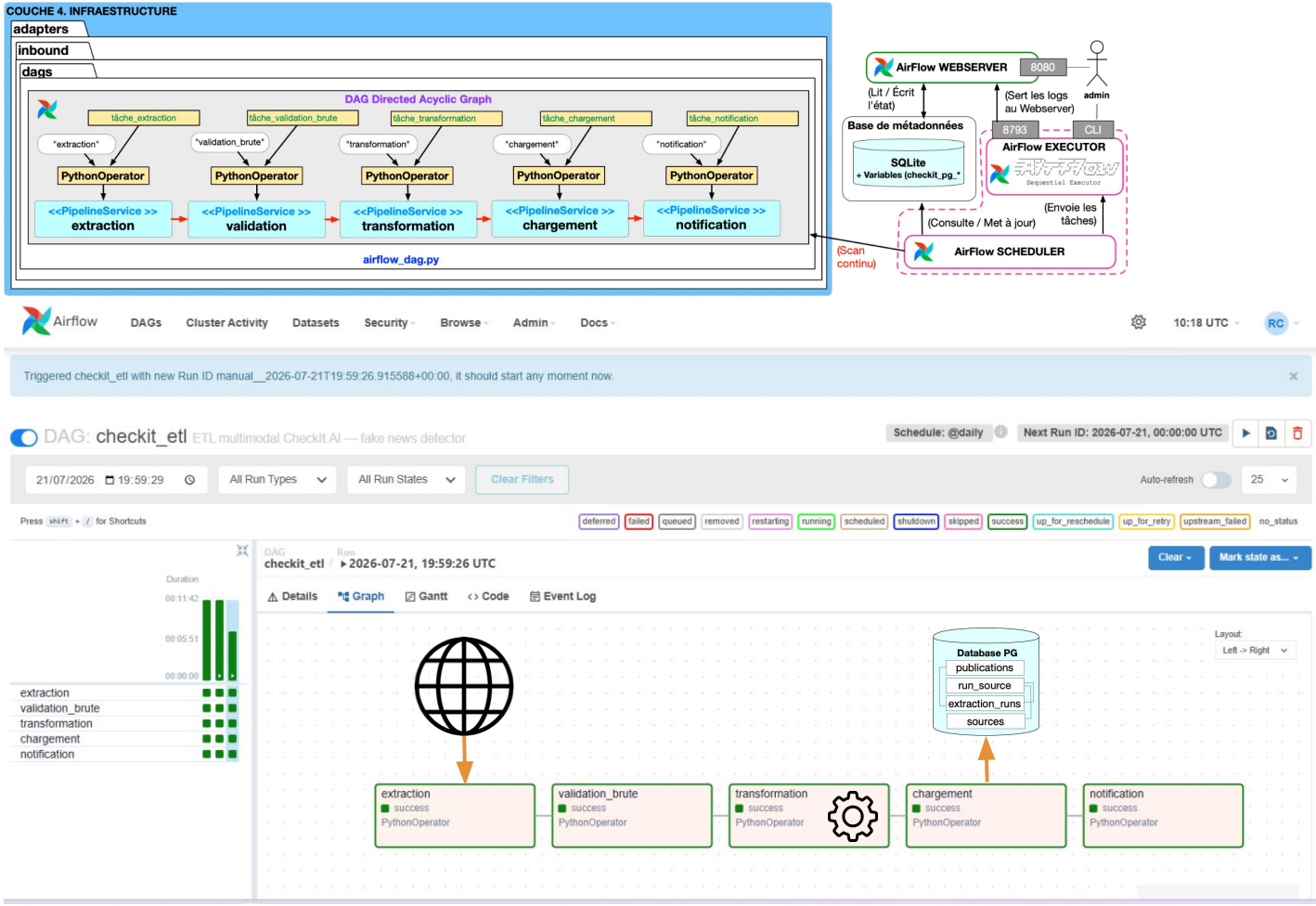
4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Étape 4. Orchestrez le pipeline avec Airflow

Pipeline complet exécutable dans Airflow



CheckIt.AI

5

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

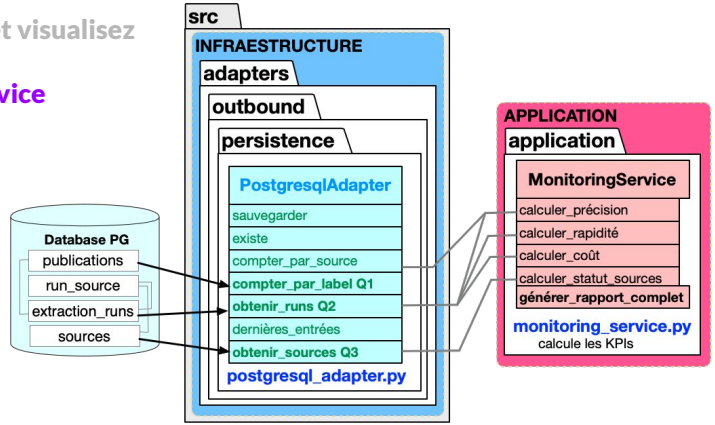
4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Étape 5 . Évaluez et visualisez les performances

⚙️Monitoring Service



Q1 — Répartition par label (compter_par_label)

Répartition REAL / FAKE du dataset complet

```
SELECT declared_label,
       COUNT(*) AS nb
FROM   publications
GROUP BY declared_label;
```

classe	effectif
FAKE	121
REAL	1159

Q2 — Runs d'extraction (obtenir_runs)

Taux d'intégrité, compteurs du DERNIER run, rapidité, coût

```
SELECT run_id,
       started_at,
       finished_at,
       nb_extraites,
       nb_valides,
       nb_rejetées,
       taux_intégrité
FROM   extraction_runs
ORDER BY started_at DESC
LIMIT  %(limite)s; -- 1 pour la précision, 20 pour rapidité/coût
```

run_id	started_at	finished_at	nb_extraites	nb_valides	nb_rejetées	taux_intégrité
run_20260721_220551	2026-07-21 22:04:47.464147+02	2026-07-21 22:05:53.654587+02	4037	1280	2757	31.7

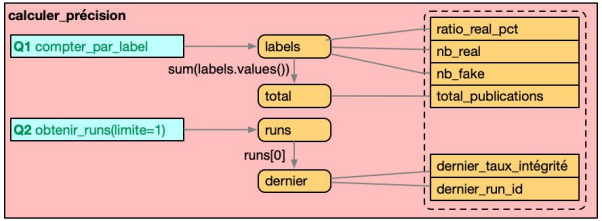
(1 row)

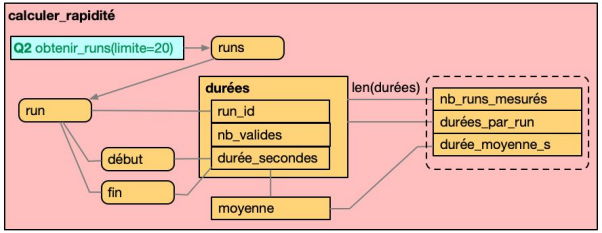
Q3 — Sources et fraîcheur (obtenir_sources)

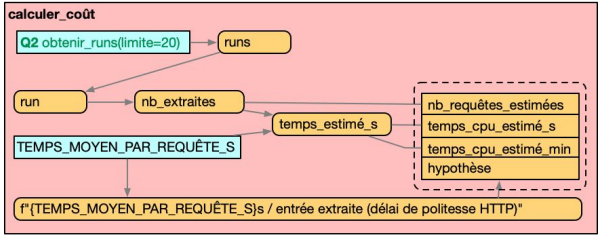
SQL réel de l'adaptateur (constante SQL_SELECT_SOURCES) :

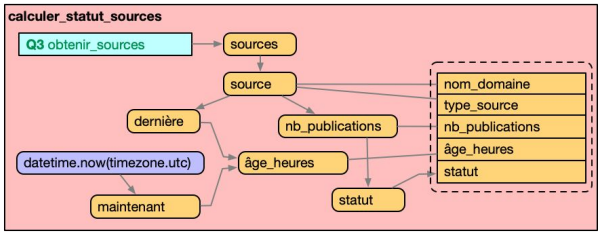
```
SELECT   nom_domaine, type_source, langue, méthode_extraction,
         première_extraction, dernière_extraction, nb_publications
FROM     sources
ORDER BY nb_publications DESC;
```

nom_domaine	type_source	langue	méthode_extraction	première_extraction	dernière_extraction	nb_publications
elpais.com	rss		feedparser	2026-07-04 07:03:27.144349+02	2026-07-21 22:05:53.655282+02	1151
politifact.com	rss		feedparser	2026-07-07 02:04:47.003911+02	2026-07-21 22:05:53.655282+02	23
correctiv.org	rss		feedparser	2026-07-04 07:03:27.144349+02	2026-07-21 22:05:53.655282+02	22
maldita.es	rss		feedparser	2026-07-04 07:03:27.144349+02	2026-07-21 22:05:53.655282+02	20
fullfact.org	rss		feedparser	2026-07-04 07:03:27.144349+02	2026-07-21 22:05:53.655282+02	14









CheckIt.AI

5

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

SQL PostgreSQL **Adapter**

requetes_verification.sql (Vérification manuelle en base: chiffres clés)

Chaque requête est l'équivalent SQL exact d'une méthode du MonitoringService (couche application) . Sert à vérifier "à la main" que le tableau de bord (console ou Streamlit) n'invente rien.

```
\echo
'--------------------------------------------------------------------------------'
\echo '1.1 Répartition REAL / FAKE du dataset complet'
\echo
'--------------------------------------------------------------------------------'
SELECT declared_label AS classe,
       COUNT(*)       AS effectif
FROM   publications
GROUP BY declared_label;
```

```
\echo
'--------------------------------------------------------------------------------'
\echo '1.2 Taux d''intégrité et compteurs du DERNIER run'
\echo
'--------------------------------------------------------------------------------'
SELECT run_id,
       started_at,
       finished_at,
       nb_extraites,
       nb_valides,
       nb_rejetées,
       taux_intégrité
FROM   extraction_runs
ORDER BY started_at DESC
LIMIT  1;
```

RÉPARTITION DATASET
1 159 REAL
121 FAKE. Ratio REAL 90,5 %

INTÉGRITÉ. DERNIER RUN
31,7 %
1 280 valides / 4 037 extraites.
2 757 rejetées

RAPIDITÉ
40,8 s moy.
sur les 15 derniers runs mesurés

COÛT ESTIMÉ
737,8 min
36 890 requêtes HTTP estimées

23/23 sources actives · traçabilité N-N (run_sources) sur les 15 derniers runs

Requêtes exécutées via requetes_verification.psq. script complet dans le livrable

Étape 5. Évaluez et visualisez les performances

Terminal DASHBOARD

console_dashboard.py

- *Consomme EXACTEMENT le même **MonitoringService** que streamlit_dashboard.py, sans aucune duplication de logique métier.*
- *Il ne fait qu'imprimer ce que le service lui retourne. La preuve concrète que remplacer (ou ajouter) un adaptateur de présentation ne coûte que ce fichier, jamais les couches inférieures.*
- *Utilité : permet de **vérifier tous les KPI en ligne de commande**, sans lancer de serveur web, avant de committer l'adaptateur Streamlit.*

```
================================================================================
             CheckIt.AI — TABLEAU DE BORD ETL (RAPPORT CONSOLE)
          Équivalent texte du livrable L6 — mêmes KPIs, même service
================================================================================

2026-07-16 16:19:24,464 [postgres] [NOTICE   ]
PostgresqlAdapter Connexion ouverte : checkit@localhost
================================================================================
SECTION 1 — PRÉCISION DES DONNÉES
================================================================================
  Publications totales.............: 1055
  REAL.............................: 939
  FAKE.............................: 116
  Ratio REAL.......................: 89.0 %
--------------------------------------------------------------------------------
  🔴 ALERTE CRITIQUE — taux d'intégrité....: 33.2 % (seuil : 70.0 %)

================================================================================
SECTION 2 — RAPIDITÉ DU PIPELINE
================================================================================
  Durée moyenne par run.............: 33.3 s
  Runs mesurés.....................: 12
--------------------------------------------------------------------------------
  Détail des 5 derniers runs mesurés :
    run_20260715_113010       .........:     1.8 s (1055 valides)
    run_20260715_111215       .........:    85.3 s (1053 valides)
    run_20260713_224203       .........:    44.7 s (925 valides)
    run_20260710_190711       .........:    40.9 s (783 valides)
    run_20260707_020445       .........:    43.4 s (627 valides)

================================================================================
SECTION 3 — COÛT ESTIMÉ (RESSOURCES)
================================================================================
  Requêtes HTTP estimées...........: 25121
  Temps CPU estimé.................: 502.4 min
--------------------------------------------------------------------------------
  Hypothèse : 1.2s / entrée extraite (délai de politesse HTTP)
```

```
================================================================================
SECTION 4 — STATUT DES SOURCES
================================================================================
  ✅ Toutes les sources sont fraîches et productives.
--------------------------------------------------------------------------------
      Source                          Type       Publications
  --------------------------------------------------------------
  [OK] elpais.com                     rss                 931
  [OK] politifact.com                 rss                  23
  [OK] correctiv.org                  rss                  22
  [OK] maldita.es                     rss                  20
  [OK] fullfact.org                   rss                  14
  [OK] euvsdisinfo.com                rss                  10
  [OK] hoaxbuster.com                 rss                   9
  [OK] afp_bluesky.com                rss                   7
  [OK] i24news_tv                     rss                   3
  [OK] lefigaro                       rss                   2
  [OK] actu                           rss                   2
  [OK] europe1                        rss                   1
  [OK] euronews_fr                    rss                   1
  [OK] laprovence                     rss                   1
  [OK] lecho                          rss                   1
  [OK] lejdd_fr                       rss                   1
  [OK] lemonde.fr                     rss                   1
  [OK] nicematin                      rss                   1
  [OK] rfi_fr                         rss                   1
  [OK] bfmtv                          rss                   1
  [OK] centrepresseaveyron            rss                   1
  [OK] charentelibre_fr               rss                   1
  [OK] estrepublicain_fr              rss                   1

================================================================================
  Rapport généré le.................: 2026-07-16T14:19:22.
================================================================================
```

monitoring_service.py

générer_rapport_complet
- généré_le

précision
- ratio_real_pct
- nb_real
- nb_fake
- total_publications
- dernier_taux_intégrité
- dernier_run_id

rapidité
- nb_runs_mesurés
- durées_par_run
- durée_moyenne_s

coût
- nb_requêtes_estimées
- temps_cpu_estimé_s
- temps_cpu_estimé_min
- hypothèse

statut_sources
- nom_domaine
- type_source
- nb_publications
- âge_heures
- statut

seuils
- intégrité_critique
- intégrité_avertissement
- fraîcheur_heures

CheckIt.AI

5

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

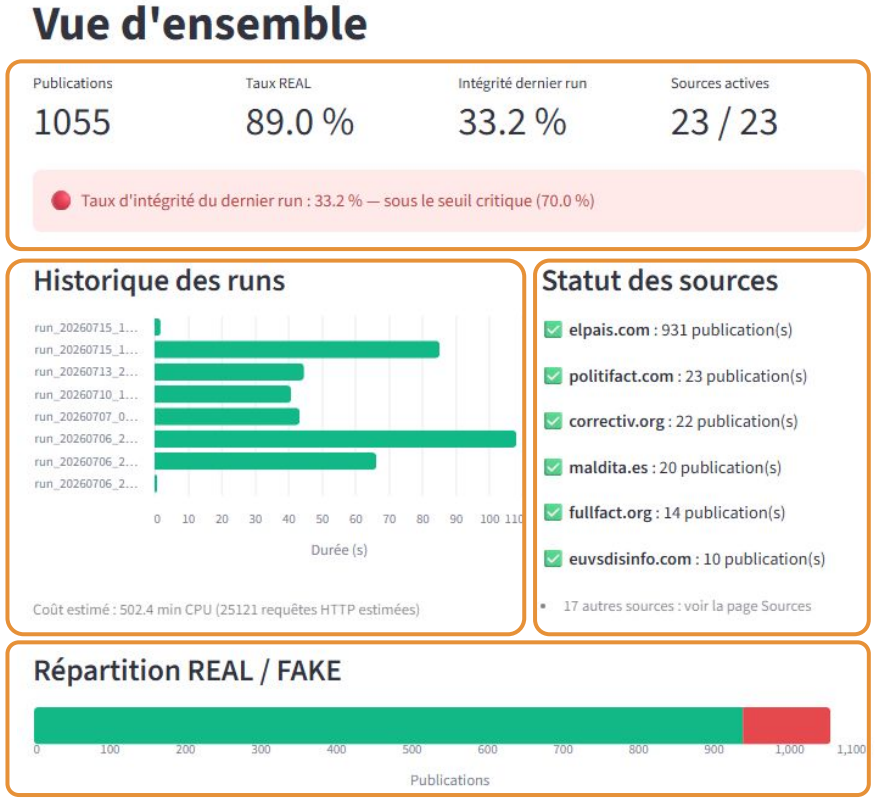
6. Conclusion

Streamlit **DASHBOARD**

Page 1 — Vue d'ensemble (page_vue_densemble)

Alerte automatique dès que le taux d'intégrité passe sous 70 %

Historique des 8 derniers runs visible en un coup d'œil



Élément affiché	Fonction de rendu (dashboard)	Fonction service	Méthode port → SQL
Métrique « Publications »	st.metric (dans page_vue_densemble)	calculer_précision()	compter_par_label() → Q1
Métrique « Taux REAL »	st.metric	calculer_précision() (ratio_real_pct, calcul Python)	compter_par_label() → Q1
Métrique « Intégrité dernier run »	st.metric	calculer_précision()	obtenir_runs(limite=1) → Q2
Métrique « Sources actives »	st.metric	calculer_statut_sources()	obtenir_sources() → Q3
Bandeau d'alerte intégrité	bandeau_intégrité() + statut_selon_taux()	calculer_précision() + seuils du rapport	obtenir_runs(limite=1) → Q2
Graphique « Historique des runs » (8 derniers)	graphique_durées() (Altair, barres horizontales)	calculer_rapidité()	obtenir_runs(limite=20) → Q2
Caption coût (min CPU, requêtes)	st.caption	calculer_coût() (proxy 1,2 s/entrée, calcul Python)	obtenir_runs(limite=20) → Q2
Liste « Statut des sources » (6 premières)	st.markdown avec ÉMOJI_STATUT	calculer_statut_sources()	obtenir_sources() → Q3
Graphique « Répartition REAL / FAKE »	graphique_répartition() (Altair, barre empilée)	calculer_précision()	compter_par_label() → Q1

CheckIt.AI

5

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Streamlit **DASHBOARD**

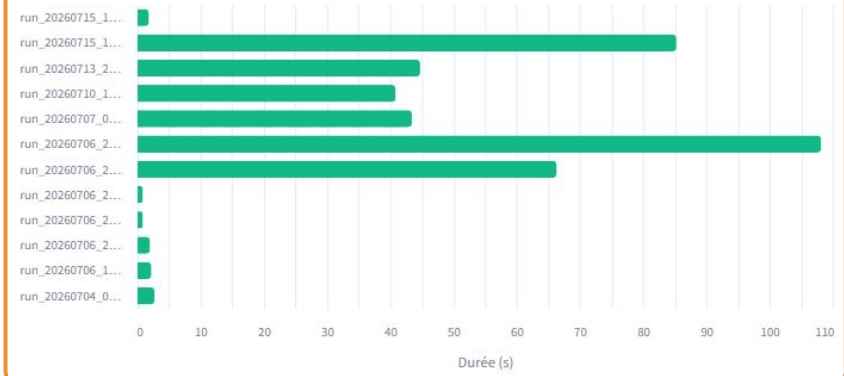
Page 2 — Rapidité (page_rapidité)

Comparaison directe avec la moyenne .
ici -31,5 s, tendance positive

Rapidité du pipeline

Durée moyenne	Dernier run	Run le plus rapide	Runs mesurés
33.3 s	1.8 s	0.8 s	12
	-31.5 s vs moyenne		

Historique complet des runs



Détail des runs récents

Run	Durée (s)	Publications valides
run_20260715_113010	1.8216	1055
run_20260715_111215	85.2523	1053
run_20260713_224203	44.7103	925
run_20260710_190711	40.8574	783

Élément affiché	Fonction de rendu	Fonction service	Méthode port → SQL
Métriques « Durée moyenne / Dernier run / Plus rapide / Runs mesurés »	st.metric × 4	calculer_rapidité() (moyenne et durées en Python)	obtenir_runs(limite=20) → Q2
Graphique « Historique complet des runs »	graphique_durées() + statut_selon_taux() (couleur	calculer_rapidité() + seuils	obtenir_runs(limite=20) → Q2
Tableau « Détail des runs récents »	st.dataframe (DataFrame pandas)	calculer_rapidité() (durées_par_run)	obtenir_runs(limite=20) → Q2

CheckIt.AI

5

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Streamlit **DASHBOARD**

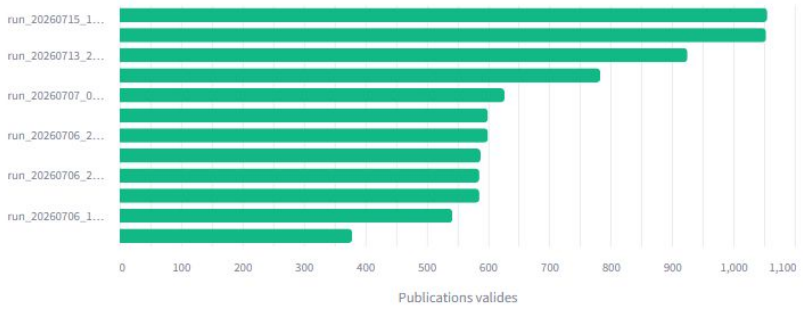
Page 3 — Coût (page_coût)

Estimation volontairement prudente, sans facturation cloud réelle sur ce projet

Coût estimé (ressources)

Requêtes HTTP estimées	Temps CPU estimé	Coût moyen / run	Coût / publication
25121	502.4 min	41.9 min	28.57 s

Publications valides par run



Hypothèse de calcul — une entrée extraite ≈ une requête HTTP (fetch RSS + éventuel fetch og:image). Formule : 1.2s / entrée extraite (délai de politesse HTTP). Estimation volontairement prudente : elle surestime plutôt qu'elle ne sous-estime le coût réel — aucune facturation cloud sur ce projet.

Élément affiché	Fonction de rendu	Fonction service	Méthode port → SQL
Métrique « Requêtes HTTP estimées »	st.metric	calculer_coût() (Σ nb_extraites)	obtenir_runs(limite=20) → Q2
Métrique « Temps CPU estimé »	st.metric	calculer_coût() (nb_extraites × 1,2 s)	obtenir_runs(limite=20) → Q2
Métrique « Coût moyen / run »	calcul local dans page_coût	calculer_coût() + calculer_rapidité()	Q2
Métrique « Coût / publication »	calcul local dans page_coût	calculer_coût() + calculer_précision()	Q2 + Q1
Graphique « Publications valides par run »	graphique_valides_par_run() (Altair, barres teal)	calculer_rapidité() (durées_par_run)	obtenir_runs(limite=20) → Q2
Encadré « Hypothèse de calcul »	st.info	calculer_coût() (hypothèse, constante TEMPS_MOYEN_PAR_REQUÊTE_S)	— (aucun SQL)

CheckIt.AI

5

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Streamlit **DASHBOARD**

Page 4 — Sources (page_sources)

23/23 sources actives. aucune alerte,
mais PolitiFact et Decodex
restent à surveiller malgré le statut vert



Élément affiché	Fonction de rendu	Fonction service	Méthode port → SQL
Métriques « Actives / En alerte / Cassées / Publications »	st.metric × 4	calculer_statut_sources() (agrégats en Python)	obtenir_sources() → Q3
Bandeaux d'alerte (rouge/orange/vert)	st.error / st.warning / st.success	calculer_statut_sources()	obtenir_sources() → Q3
Grille de cartes par source (liseré coloré)	carte_source_html() + st.markdown	calculer_statut_sources() (tri rouge→orange→vert)	obtenir_sources() → Q3

Conclusion

CheckIt.AI

6

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

⚠️**Risques identifiés à surveiller**

Qualité des données (risques hors disponibilité)

- **Dérive du taux REAL/FAKE**: déjà identifié dans les notes comme déséquilibre historique vers FAKE
- **Doublons non détectés malgré le hash SHA-256**: si le contenu change légèrement entre deux extractions du même article
- **Labels non normalisables en hausse**: ex. Chequeado avec 50 entrées rejetées pour 'aucun mot-clé'
- **Association texte-image rompue**: image_url valide au moment de l'extraction mais supprimée avant la validation

Infrastructure

- **Contention WSL2/SQLite**: le DagFileProcessorManager qui perd son heartbeat, déjà observé.
- **PID file webserver obsolète** après suspension WSL2 sans arrêt propre
- **Variable Airflow manquante** après régénération de la base de métadonnées

Conclusion

CheckIt.AI

6

i. Introduction
- Présentation
- Architecture du projet
- Schéma de données
- Modèle conceptuel / physique

1. Sources de données
- Sources qualifiées

2. Scripts d'extraction
- Scrapers

3. Transformation pipeline
- ⚙️*TransformService*
- ⚙️*Pipeline Service*

4. Orchestration Airflow
- DAG checkit_etl

5. Performances
- ⚙️*Monitoring Service*
- Requêtes Vérification
- Console Dashboard
- Streamlit Dashboard

6. Conclusion

Plan de monitoring: stratégie de surveillance du pipeline en production.

Logging

- **Module logging natif** : handlers fichier + console
- Niveaux **DEBUG** → **ERROR** selon l'environnement
- **Format** : timestamp · niveau · module · message
- **Rotation** : 7 jours, 10 Mo max par fichier

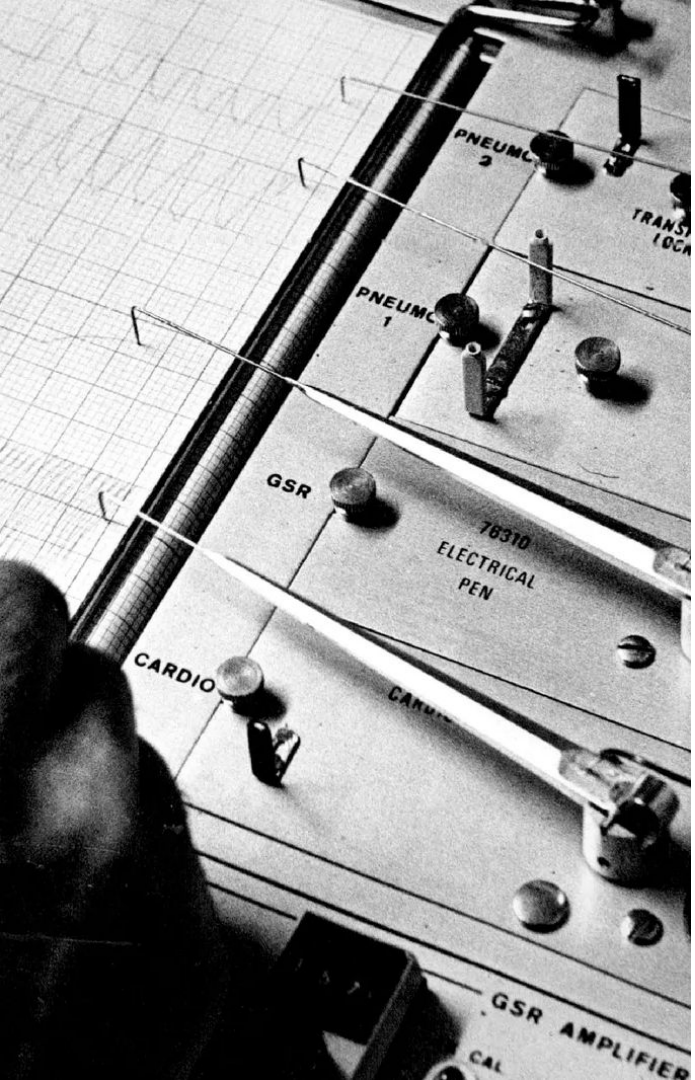
Seuils d'alerte

- **CRITIQUE**: intégrité < 70 % → arrêt + email
- **AVERTISSEMENT**: quota API > 80 %
- **INFO:** rapport quotidien à 08:00 UTC (schedule = "0 8 * * *")

Fréquence de vérification

- **Temps réel:** erreurs HTTP, exceptions
- **5 min:** disponibilité des sources (Seconde DAG ou CRON)
- **Quotidien:** rapport KPIs complet
- **Hebdomadaire:** audit qualité des labels

Constat en production: le dernier run mesuré affiche un taux d'intégrité de 31,7 %, sous le seuil critique de 70 % défini dans ce plan. L'alerte **CRITIQUE** se serait déclenchée automatiquement : le plan de monitoring fonctionne comme prévu face à une dégradation réelle des sources.



Q et R

Rafael CEREZO MARTIN

Juin 2026